Vel Tech
Rangarajan Dr. Sagunthala
R&D Institute of Science and Technology
DEEMED TO BE
University
(Estd. u/s 3 of UGC Act, 1956)
Avadi, Chennai

School of Computing

Department of Computer Science and Engineering

USE CASE REAL LIFE EXAMPLE

Program- Department Name	B.Tech – CSE
Academic Year/Semester	2026-27 / Summer
Faculty Name	Dr. S. Hemamalini
Course Code/Course Name	10211CS227 / Problem solving and Testing
Slot No & Date	S3-2 / 22-10-2026
Student Name	K.Hrushikesh Reddy
Title of the Complex Engineering problem / problems	SUPER MARKET BILLING SYSTEM
Description / Complexity Addressed	The Super Market Billing System addresses the challenges of running a retail counter: keeping a catalogue of products identified by unique codes, restocking items, preparing itemised customer invoices, and keeping the stock count correct after every sale. The system supports member discounts, adds GST to the final amount, raises a low-stock alert, and produces a daily sales report. It validates every input, blocks sales of unknown products or quantities beyond the available stock, and rejects invalid entries. It demonstrates the use of classes, objects, collections, methods, loops and conditional statements.
CO's Mapped	CO1, CO2, CO3, CO4, CO5
PO's / PSO's Mapped	PO1, PO2, PO3, PO5, PSO2
SDG Mapped with number	SDG 8: Decent Work and Economic Growth.
Justification for SDG Mapped	The Super Market Billing System supports SDG 8 – Decent Work and Economic Growth by improving the productivity and reliability of small and medium retail businesses. Automated invoicing removes manual calculation work, shortens customer waiting time, and keeps tax and discount calculations consistent. Up-to-date stock and sales records help shop owners plan purchases and run their business efficiently.
Proof for the solved Complex Engineering Problem	https://drive.google.com/drive/folders/1xRIlDrZpvEvbM77e1Vs3WojLz105Gwje?usp=drive_link

Signature of the Faculty

TABLE OF CONTENTS

S. No.	Contents	Page No.
1.	Use-case	3-4
2.	Problem Statement	5
3.	Code	6-9
4.	Output Screenshot	10-11
5.	Result	12-14

1.CASE STUDY:

The Super Market Billing System is a console-based Java application that helps a store handle everyday counter work in one place. The cashier registers products using unique product codes, restocks items when they run low, bills customers with an itemised invoice, and reviews the stock list and the sales of the day. The system applies a membership discount, adds GST automatically, and warns when a product is about to run out, so that billing stays quick and error free.

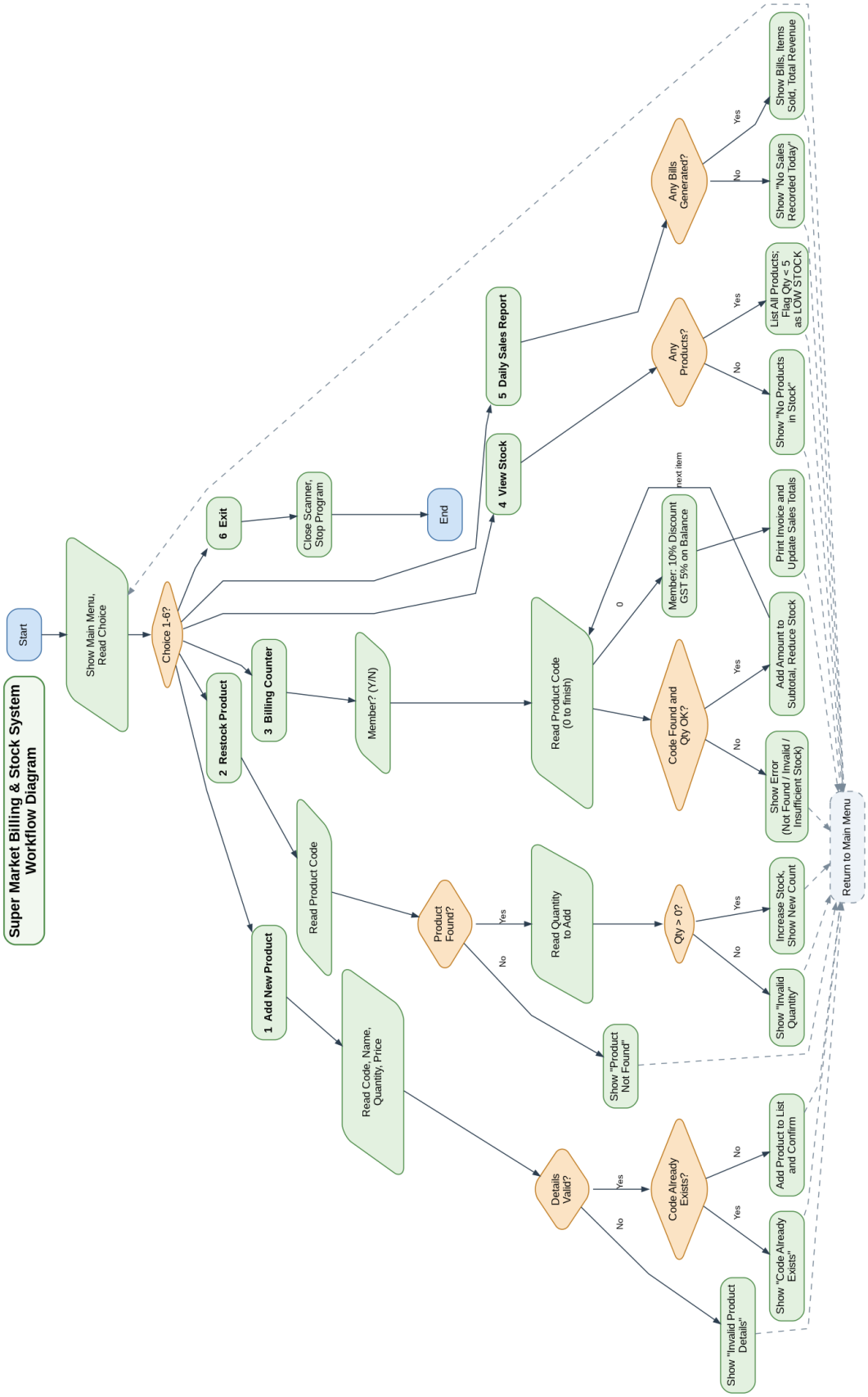
The system demonstrates important programming concepts such as:

- Object-Oriented Design: A Product class stores the code, name, quantity and price of every item.
- Product Catalogue: Adding products with unique codes and restocking existing products.
- Billing Counter: Reading items in a loop and building an itemised invoice with a running subtotal.
- Discount and Tax: A 10% member discount followed by 5% GST on the discounted amount.
- Stock Control: Reducing stock after each sale and flagging products with fewer than 5 units as low stock.
- Validation and Error Handling: Handling invalid numbers, unknown codes, duplicate codes, excess quantity and wrong menu choices.

Purpose of the System

The purpose of the Super Market Billing System is to give small and medium stores an easy-to-use tool for preparing accurate invoices and tracking their stock. It saves the cashier's time, avoids mistakes in discount and tax calculation, and gives the owner a clear view of stock levels and daily sales.

Block Diagram:



2.Problem Statement:

In many small stores, bills are still prepared by hand or with a calculator. The cashier has to look up prices, multiply quantities, work out the member discount and the GST, and then separately update the stock register. This is slow, and it often leads to wrong totals, items sold beyond the available stock, and stock records that do not match the shelves.

The proposed system solves this with a menu-driven application that stores the product catalogue, prepares invoices automatically, updates the stock after every sale, and reports the sales of the day.

Functional Requirements:

- Add New Product: Register a product with a unique code, name, quantity and price.
- Restock Product: Increase the stock of an existing product using its product code.
- Billing Counter: Read products and quantities for a customer and prepare an itemised invoice.
- Member Discount and GST: Give a 10% discount to members and add 5% GST on the discounted amount.
- View Stock: List all products with their quantity and price and mark products with fewer than 5 units as low stock.
- Daily Sales Report: Display the number of bills generated, the items sold and the total revenue.
- Input Validation: Reject empty or non-numeric entries, zero or negative values, duplicate product codes, unknown products and quantities above the available stock.
- Menu Validation: Display an error message when the user selects an invalid menu option.
- Exit: Terminate the application safely.

Complexity Addressed

The system addresses problems involving a product catalogue, repeated item entry during billing, combined discount and tax calculation, live stock updates and input validation. It demonstrates the use of classes and objects, collections, methods, conditional statements, loops and exception handling.

Expected Outcome

The system should produce correct itemised invoices, keep the stock count up to date, warn about low-stock products, reject invalid transactions and report the sales of the day. This improves the speed, accuracy and reliability of the billing counter.

3. Code:

```
import java.util.*;

public class Main {

    static class Product {
        String code;
        String name;
        int qty;
        double price;

        Product(String code, String name, int qty, double price) {
            this.code = code;
            this.name = name;
            this.qty = qty;
            this.price = price;
        }
    }

    static Scanner sc = new Scanner(System.in);
    static ArrayList<Product> products = new ArrayList<>();

    static final double GST_RATE = 0.05;
    static final double MEMBER_DISCOUNT = 0.10;
    static final int LOW_STOCK_LIMIT = 5;

    static int invoiceNo = 1000;
    static int billsGenerated = 0;
    static int itemsSold = 0;
    static double revenue = 0;

    static String readText(String prompt) {
        System.out.print(prompt);
        return sc.nextLine().trim();
    }

    static int readInt(String prompt) {
        try {
```

```
            return Integer.parseInt(readText(prompt));
        } catch (NumberFormatException e) {
            return -1;
        }
    }

    static double readDouble(String prompt) {
        try {
            return Double.parseDouble(readText(prompt));
        } catch (NumberFormatException e) {
            return -1;
        }
    }

    static Product findProduct(String code) {
        for (Product p : products) {
            if (p.code.equalsIgnoreCase(code)) {
                return p;
            }
        }
        return null;
    }

    public static void main(String[] args) {
        while (true) {
            System.out.println("\n===== SUPER MARKET BILLING &
            STOCK SYSTEM =====");
            System.out.println("1. Add New Product");
            System.out.println("2. Restock Product");
            System.out.println("3. Billing Counter");
            System.out.println("4. View Stock");
            System.out.println("5. Daily Sales Report");
            System.out.println("6. Exit");
            int choice = readInt("Enter Choice: ");

            switch (choice) {

                // 1. Add New Product
```

```
case 1: {
    String code = readText("Enter Product Code:
    ").toUpperCase();
    String name = readText("Enter Product Name: ");
    int qty = readInt("Enter Quantity: ");
    double price = readDouble("Enter Price: ");

    if (code.isEmpty() || name.isEmpty() || qty <= 0 || price <= 0)
    {
        System.out.println("Invalid Product Details");
    } else if (findProduct(code) != null) {
        System.out.println("Product Code Already Exists");
    } else {
        products.add(new Product(code, name, qty, price));
        System.out.println("Product Added: " + name + " (" +
        code + ")");
    }
    break;
}

// 2. Restock Product
case 2: {
    String code = readText("Enter Product Code: ");
    Product p = findProduct(code);
    if (p == null) {
        System.out.println("Product Not Found");
        break;
    }
    int add = readInt("Enter Quantity to Add: ");
    if (add <= 0) {
        System.out.println("Invalid Quantity");
    } else {
        p.qty += add;
        System.out.println("Stock Updated. " + p.name + " now
        has " + p.qty + " units");
    }
    break;
}
```

```
// 3. Billing Counter
case 3: {
    String member = readText("Is the customer a member?
    (Y/N): ").toUpperCase();
    if (!member.equals("Y") && !member.equals("N")) {
        System.out.println("Invalid Input. Enter Y or N");
        break;
    }

    ArrayList<String> lines = new ArrayList<>();
    double subtotal = 0;
    int units = 0;

    while (true) {
        String code = readText("Enter Product Code (0 to
        finish): ");
        if (code.equals("0")) {
            break;
        }
        Product p = findProduct(code);
        if (p == null) {
            System.out.println("Product Not Found");
            continue;
        }
        int qty = readInt("Enter Quantity: ");
        if (qty <= 0) {
            System.out.println("Invalid Quantity");
            continue;
        }
        if (qty > p.qty) {
            System.out.println("Insufficient Stock. Available: " +
            p.qty);
            continue;
        }
        double amount = qty * p.price;
        p.qty -= qty;
        subtotal += amount;
```

```
            units += qty;
            lines.add(String.format("%-8s x%-3d Rs. %8.2f",
            p.name, qty, amount));
        }

        if (lines.isEmpty()) {
            System.out.println("No Items Purchased");
            break;
        }

        double discount = member.equals("Y") ? subtotal *
        MEMBER_DISCOUNT : 0;
        double gst = (subtotal - discount) * GST_RATE;
        double grandTotal = subtotal - discount + gst;

        invoiceNo++;
        billsGenerated++;
        itemsSold += units;
        revenue += grandTotal;

        System.out.println("\n----- INVOICE #" + invoiceNo + " ----
        -");
        for (String line : lines) {
            System.out.println(line);
        }
        System.out.printf("Subtotal   : Rs. %.2f%n", subtotal);
        System.out.printf("Discount   : Rs. %.2f%n", discount);
        System.out.printf("GST (5%%)   : Rs. %.2f%n", gst);
        System.out.printf("Grand Total: Rs. %.2f%n", grandTotal);
        break;
    }

    // 4. View Stock
    case 4: {
        if (products.isEmpty()) {
            System.out.println("No Products in Stock");
        } else {
            System.out.println("\n----- STOCK LIST -----");
```

```
                    for (Product p : products) {
                        String alert = p.qty < LOW_STOCK_LIMIT ? "  <--
                        LOW STOCK" : "";
                        System.out.printf("%s | %s | Qty: %d | Rs.
                        %.2f%s%n",
                                p.code, p.name, p.qty, p.price, alert);
                    }
                }
                break;
            }

            // 5. Daily Sales Report
            case 5: {
                if (billsGenerated == 0) {
                    System.out.println("No Sales Recorded Today");
                } else {
                    System.out.println("\n----- DAILY SALES REPORT ----
                    -");
                    System.out.println("Bills Generated: " + billsGenerated);
                    System.out.println("Items Sold     : " + itemsSold);
                    System.out.printf("Total Revenue  : Rs. %.2f%n",
                    revenue);
                }
                break;
            }

            // 6. Exit
            case 6:
                System.out.println("Thank You. Application Closed");
                sc.close();
                return;

            default:
                System.out.println("Invalid Choice. Enter 1 to 6");
        }
    }
}
```

4. OUTPUT SCREENSHOT

Output

```
1. Add New Product
2. Restock Product
3. Billing Counter
4. View Stock
5. Daily Sales Report
6. Exit
Enter Choice:
```

Output

```
1. Add New Product
2. Restock Product
3. Billing Counter
4. View Stock
5. Daily Sales Report
6. Exit
Enter Choice: 1
Enter Product Code: P101
Enter Product Name: Rice
Enter Quantity: 20
Enter Price: 60
Product Added: Rice (P101)
```

Output

```
1. Add New Product
2. Restock Product
3. Billing Counter
4. View Stock
5. Daily Sales Report
6. Exit
Enter Choice: 3
Is the customer a member? (Y/N): N
Enter Product Code (0 to finish): P101
Enter Quantity: 2
Enter Product Code (0 to finish): P102
Enter Quantity: 1
Enter Product Code (0 to finish): 0

----- INVOICE #1001 -----
Rice     x2   Rs.   120.00
Sugar    x1   Rs.    45.00
Subtotal   : Rs. 165.00
Discount   : Rs. 0.00
GST (5%)   : Rs. 8.25
Grand Total: Rs. 173.25

1. Add New Product
2. Restock Product
3. Billing Counter
4. View Stock
5. Daily Sales Report
6. Exit
Enter Choice:
```

5.Result :

The Super Market Billing System was designed to manage the product catalogue, prepare customer invoices, apply member discount and GST, update stock automatically and report the daily sales. The system validates user inputs, handles unknown products and excess quantities, and calculates the grand total accurately. Testing with valid and invalid inputs helps verify the correctness and reliability of the application. Thus, the objective of developing a simple and efficient Super Market Billing System using Java programming concepts was achieved.

Table I: Test Cases and Execution Results for the Super Market Billing System

Test Case ID	Choice	Test Scenario	Input / Action	Expected Output	Execution Output (Copy the execution sequence of this choice)	Status
TC01	1	Valid Product Addition	Code: P101, Name: Rice, Qty: 20, Price: 60	Product should be added to the catalogue	Enter Choice: 1 Enter Product Code: P101 Enter Product Name: Rice Enter Quantity: 20 Enter Price: 60 Product Added: Rice (P101)	Pass
TC02	1	Invalid Price	Code: P101, Name: Rice, Qty: 20, Price: -10	System should reject invalid product details	Enter Choice: 1 Enter Product Code: P101 Enter Product Name: Rice Enter Quantity: 20 Enter Price: -10 Invalid Product Details	Pass
TC03	1	Duplicate Product Code	Code: P101 (already added)	System should reject the duplicate code	Enter Choice: 1 Enter Product Code: P101 Enter Product Name: Rice Enter Quantity: 20 Enter Price: 60 Product Code Already Exists	Pass
TC04	2	Valid Restock	Code: P101, Quantity to add: 10	Stock should increase and the new count be shown	Enter Choice: 2 Enter Product Code: P101 Enter Quantity to Add: 10 Stock Updated. Rice now has 30 units	Pass
TC05	2	Restock Unknown Product	Code: P999	System should display product not found message	Enter Choice: 2 Enter Product Code: P999 Product Not Found	Pass
TC06	3	Bill for Non-Member	Rice x2 (Rs. 60), Sugar x1 (Rs. 45), Member: N	Invoice with 5% GST and no discount should be generated	Enter Choice: 3 Is the customer a member? (Y/N): N Enter Product Code (0 to finish): P101 Enter Quantity: 2 Enter Product Code (0 to finish): P102 Enter Quantity: 1 Enter Product Code (0 to finish): 0 ----- INVOICE #1001 ----- Rice x2 Rs. 120.00 Sugar x1 Rs. 45.00 Subtotal : Rs. 165.00 Discount : Rs. 0.00	Pass

Test Case ID	Choice	Test Scenario	Input / Action	Expected Output	Execution Output (Copy the execution sequence of this choice)	Status
					GST (5%) : Rs. 8.25 Grand Total: Rs. 173.25	
TC07	3	Bill for Member	Rice x10 (Rs. 60), Member: Y	10% discount and 5% GST should be applied	Enter Choice: 3 Is the customer a member? (Y/N): Y Enter Product Code (0 to finish): P101 Enter Quantity: 10 Enter Product Code (0 to finish): 0 ----- INVOICE #1001 ----- Rice x10 Rs. 600.00 Subtotal : Rs. 600.00 Discount : Rs. 60.00 GST (5%) : Rs. 27.00 Grand Total: Rs. 567.00	Pass
TC08	3	Insufficient Stock	Rice x50 (only 20 in stock)	System should reject quantity above available stock	Enter Choice: 3 Is the customer a member? (Y/N): N Enter Product Code (0 to finish): P101 Enter Quantity: 50 Insufficient Stock. Available: 20 Enter Product Code (0 to finish): 0 No Items Purchased	Pass
TC09	4	View Stock with Low-Stock Alert	Select Choice 4 (Soap has 3 units)	All products should be listed and Soap flagged as low stock	Enter Choice: 4 ----- STOCK LIST ----- P101 | Rice | Qty: 20 | Rs. 60.00 P102 | Sugar | Qty: 10 | Rs. 45.00 P103 | Soap | Qty: 3 | Rs. 30.00 <-- LOW STOCK	Pass
TC10	4	Empty Stock	Select Choice 4 when no products are added	System should indicate that no products are in stock	Enter Choice: 4 No Products in Stock	Pass
TC11	5	Daily Sales Report	Select Choice 5 after one bill	Bills, items sold and total revenue should be displayed	Enter Choice: 5 ----- DAILY SALES REPORT ----- Bills Generated: 1 Items Sold : 3 Total Revenue : Rs. 173.25	Pass
TC12	5	No Sales	Select Choice 5 before any bill	System should display that no sales are recorded	Enter Choice: 5 No Sales Recorded Today	Pass
TC13	Invalid	Invalid Menu Choice	Enter Choice: 9	System should reject invalid menu option	Enter Choice: 9 Invalid Choice. Enter 1 to 6	Pass
TC14	6	Exit Application	Select Choice 6	Application should terminate successfully	Enter Choice: 6 Thank You. Application Closed	Pass